TrustGRC AI 360

Live Platform Architecture, Database Schema, URLs and Operating Guide

Unified AI Governance, Risk & Compliance Platform

```
CURRENT DEPLOYMENT STATUS
GitHub repository  -> synchronized
Hostman VPS        -> online
FastAPI API        -> online as systemd service
PostgreSQL         -> connected and verified
Swagger/OpenAPI    -> publicly accessible
Database tables    -> organizations, ai_systems, generated_risks
```

Version 0.1.0 | 31 July 2026

Document Contents

1. Executive Summary

TrustGRC AI 360 is now operating as a live cloud-hosted MVP foundation. The backend is deployed on a Hostman Ubuntu VPS, runs through FastAPI and Uvicorn, connects to a managed PostgreSQL database, and is maintained as a persistent systemd service. The source code is synchronized with GitHub. Public API health and Swagger documentation endpoints have been tested successfully from a browser.

The current platform foundation supports the following core workflow:

- Create an organisation
- Register an AI system
- Generate governance, compliance and cybersecurity risks
- Store the generated risks in PostgreSQL
- Display risks, controls and compliance information through dashboards and reports

Important current-state note: The backend API is publicly live. The Next.js frontend has been developed locally but has not yet been confirmed as publicly deployed on a permanent URL. Therefore, frontend URLs in this document are separated into current local options and recommended production options.

2. Current Live Environment

Component	Current value	Status / purpose
Product	TrustGRC AI 360	Unified AI governance, risk and compliance platform
GitHub repository	github.com/Soobi1008/trustgrc-360-ai	Source code synchronized with VPS
VPS	trustgrc360-api	Hostman Ubuntu server in Frankfurt
VPS public IP	195.216.168.137	Public access to the backend API
Backend framework	FastAPI + Uvicorn	REST API and OpenAPI documentation
Backend service	trustgrc-api.service	Runs continuously through systemd
API port	8000	Current direct public API port
Database cluster	trustgrc360-db	Managed Hostman PostgreSQL cluster
Database	default_db	Application database
Database user	gen_user	Application database account
PostgreSQL version	18	Managed relational database
Database tables	organizations, ai_systems, generated_risks	Current application data model

2.1 Live Backend URLs

Purpose	URL	Expected result
API root	http://195.216.168.137:8000/	Application name, message and version
Health check	http://195.216.168.137:8000/health	{"status":"healthy","service":"trustgrc-api"}
Swagger UI	http://195.216.168.137:8000/docs	Interactive API documentation and testing
OpenAPI JSON	http://195.216.168.137:8000/	Machine-readable API contract

	openapi.json	
Alternative docs	http://195.216.168.137:8000/redoc	ReDoc API documentation, if enabled by FastAPI defaults

3. Platform Architecture

```
User / Browser
      |
      v
Next.js Frontend
      |
      | HTTP / JSON REST requests
      v
FastAPI Backend on Hostman VPS
      |
      | SQLAlchemy ORM + psycopg driver
      v
Hostman PostgreSQL Database
      |
      v
Risk Engine / Compliance Engine
      |
      v
Dashboards, controls, evidence and reports
```

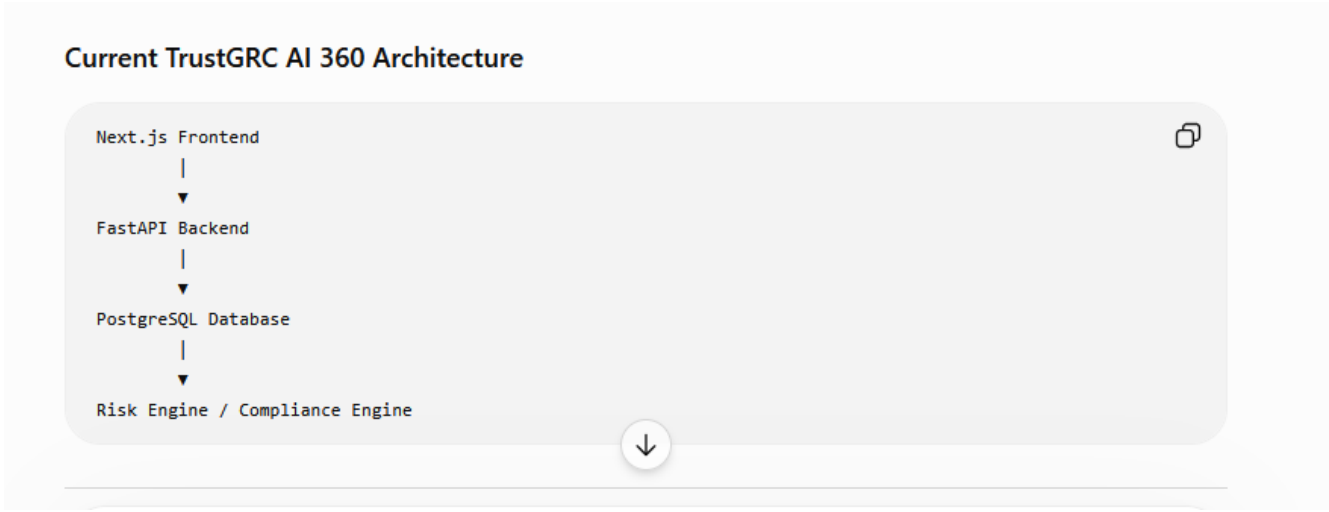

Figure 1. Current TrustGRC AI 360 architecture supplied during the deployment session.

3.1 Component Responsibilities

Layer	Responsibilities
Next.js frontend	User interface, forms, dashboards, navigation, validation, API calls and report views.
FastAPI backend	Business logic, request validation, risk generation, database operations, API routing and OpenAPI documentation.
SQLAlchemy	Maps Python models to PostgreSQL tables and manages

	queries and relationships.
PostgreSQL	Persistent storage for organisations, AI systems, generated risks and future governance artefacts.
Risk / compliance engine	Evaluates AI-system attributes and creates mapped risks, controls and regulatory references.
systemd	Starts the API automatically after reboot and restarts it if the process fails.
GitHub	Source control, version history, backup and future deployment collaboration.

4. End-to-End Business Workflow

```
Create Organization
        |
        v
Register AI System
        |
        v
Generate Risks
        |
        v
Store Risks in PostgreSQL
        |
        v
Dashboard, Review and Reports
```

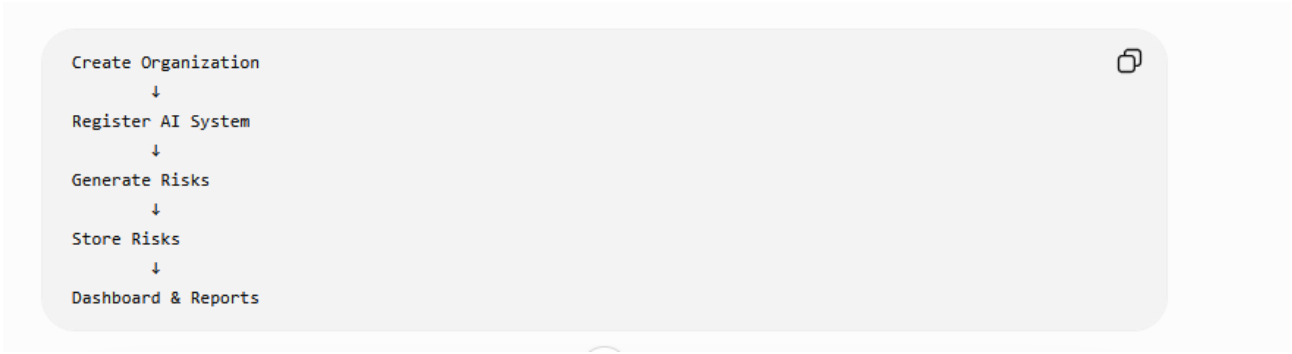

Figure 2. Proposed end-to-end TrustGRC AI 360 user workflow.

Step	User action	System action	Stored result
1	Create organisation	Validates details and creates the organisation record	organizations
2	Register AI system	Associates the system with an organisation	ai_systems
3	Select Generate Risks	Applies risk rules or future AI analysis	Generated risk objects
4	Confirm/store results	Saves risks linked to the AI system	generated_risks
5	Review dashboard	Aggregates risk scores, categories and regulations	Dashboard and reports

5. Database Schema and Relationships

```
organizations (1)
      |
      | one organisation has many AI systems
      v
ai_systems (many)
      |
      | one AI system has many generated risks
      v
generated_risks (many)

Deleting an AI system removes its related generated risks
because the foreign key uses ON DELETE CASCADE.
```

5.1 organisations Table

Purpose: stores tenant or customer organisation information. The exact columns should be verified against the current SQLAlchemy model before production documentation is finalised. Typical fields include organisation ID, name, industry, size, region and timestamps.

5.2 ai_systems Table

Purpose: provides the AI inventory. It stores each AI system assessed by the platform and links the system to its organisation. Typical attributes may include name, vendor, purpose, department, AI type, deployment status, owner, data categories and risk classification.

5.3 generated_risks Table - Verified Live Schema

Column	Type	Null?	Purpose
id	integer	No	Primary key
ai_system_id	integer	No	Foreign key to ai_systems.id
title	varchar(255)	No	Short risk title
category	varchar(100)	No	Risk category
description	text	No	Detailed risk description
reason_generated	text	Yes	Why the platform generated the risk
likelihood	varchar(50)	No	Likelihood rating
impact	varchar(50)	No	Impact rating
risk_score	integer	No	Calculated risk score
recommended_control	text	No	Recommended mitigating control
regulation	varchar(255)	Yes	Mapped framework or regulation
generation_source	varchar(100)	No	Rule, engine or AI source
review_status	varchar(50)	No	Workflow review status
created_at	timestamp with time zone	No	Record creation timestamp

Verified indexes and constraint:

- Primary key: generated_risks_pkey on id
- Index: ix_generated_risks_ai_system_id on ai_system_id
- Index: ix_generated_risks_id on id
- Foreign key: generated_risks.ai_system_id -> ai_systems.id
- Delete behaviour: ON DELETE CASCADE

6. Backend API and Public URL Options

FastAPI automatically provides interactive documentation. The Swagger interface can be used to inspect routes, view request/response schemas and execute test requests directly from a browser.

Environment	Base URL	Use
Current production-like VPS	http://195.216.168.137:8000	Live public backend currently tested
Local backend	http://127.0.0.1:8000 or http://localhost:8000	Development on the same computer
Future domain with HTTPS	https://api.trustgrc360.com	Recommended production pattern after domain and TLS configuration

6.1 Existing Confirmed Routes

Route	Method	Purpose
/	GET	Application identity and version
/health	GET	Health monitoring
/docs	GET	Swagger UI
/openapi.json	GET	OpenAPI specification
/api/v1/version	GET	Version endpoint previously configured
Organisation router routes	Multiple	Create, retrieve and manage organisations
AI-system router routes	Multiple	Create, retrieve and manage AI systems
Intelligence / generated-risk routes	Multiple	Risk-generation functionality and results

Because route paths inside each router can change during development, Swagger at /docs is the authoritative current list. The platform document should be updated whenever routes are renamed or versioned.

7. Frontend Options and Connection Strategy

Frontend option	Example URL	When to use
Local Next.js development	http://localhost:3000	Normal local development
Alternative local port	http://localhost:3001	Used when port 3000 is occupied
Hostman/VPS frontend	http://195.216.168.137:3000	Possible temporary deployment, but not recommended as the final public arrangement
Production domain	https://app.trustgrc360.com	Recommended final user-facing URL

Single-domain reverse proxy	https://trustgrc360.com	Frontend at / and backend proxied under /api

7.1 Recommended Frontend Environment Variable

```
# frontend/.env.local for development
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000

# production frontend environment
NEXT_PUBLIC_API_BASE_URL=https://api.trustgrc360.com
```

Until HTTPS and a domain are configured, the live test value can be:

```
NEXT_PUBLIC_API_BASE_URL=http://195.216.168.137:8000
```

The frontend should not hard-code the backend address in many files. It should read one environment variable through a shared API client module. This makes local, test and production deployments easier to manage.

8. Backend Operations and Service Management

Task	VPS command
Check service status	sudo systemctl status trustgrc-api
Restart after code/config changes	sudo systemctl restart trustgrc-api
Stop service	sudo systemctl stop trustgrc-api
Start service	sudo systemctl start trustgrc-api
Enable at boot	sudo systemctl enable trustgrc-api
Recent service logs	sudo journalctl -u trustgrc-api -n 100 --no-pager
Follow live logs	sudo journalctl -u trustgrc-api -f
Local health test	curl http://127.0.0.1:8000/health

8.1 Current systemd Service Design

```
[Unit]
Description=TrustGRC AI 360 FastAPI Backend
After=network.target

[Service]
User=root
WorkingDirectory=/opt/trustgrc-360-ai/backend
EnvironmentFile=/opt/trustgrc-360-ai/backend/.env
ExecStart=/opt/trustgrc-360-ai/backend/.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port
8000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Production hardening recommendation: later replace User=root with a dedicated non-root service account, place Nginx or Caddy in front of Uvicorn, and expose only ports 80/443 publicly.

9. Security and Configuration Controls

- Never commit backend/.env to GitHub. It contains database credentials.
- Keep the database password URL-encoded inside DATABASE_URL when special characters are used.
- Rotate the GitHub personal access token and database password if either is exposed.
- Restrict PostgreSQL network access to the VPS private IP whenever Hostman rules permit.
- Add HTTPS before real customer or personal data is used.
- Use a dedicated application user rather than root for the systemd service.
- Introduce authentication, role-based access control and organisation-level tenant isolation before customer use.
- Replace Base.metadata.create_all() with Alembic migrations as the schema grows.
- Add backups, restore tests, audit logs, rate limiting and security headers.

10. Deployment and Update Workflow

```
LOCAL DEVELOPMENT
Edit code -> test locally -> git add -> git commit -> git push

VPS UPDATE
SSH/login -> cd /opt/trustgrc-360-ai -> git pull origin main
-> activate virtual environment -> install requirements if changed
-> restart trustgrc-api -> test /health and /docs
```

Step	Command / action
1	cd /opt/trustgrc-360-ai
2	git pull origin main
3	cd backend && source .venv/bin/activate
4	pip install -r requirements.txt when dependencies change
5	sudo systemctl restart trustgrc-api
6	sudo systemctl status trustgrc-api
7	curl http://127.0.0.1:8000/health
8	Open http://195.216.168.137:8000/docs in a browser

11. MVP Demonstration Scenario

A strong demonstration should show one complete business journey rather than many disconnected pages.

Demo stage	Example
Organisation	Create “Example Health Services GmbH”

AI system	Register “Clinical Triage Assistant”
System attributes	Healthcare, NLP, patient data, decision support, production pilot
Generate risks	Prompt injection, sensitive-data leakage, hallucination, bias and unauthorised access
Map frameworks	GDPR, EU AI Act, ISO/IEC 42001 and NIST AI RMF
Recommend controls	Human oversight, access control, logging, data minimisation and testing
Dashboard	Show total risks, high/medium/low counts and compliance score
Report	Export or display an AI risk register

12. Recommended Next Development Steps

1. Connect the frontend to the live API - Create a shared API client and configure NEXT_PUBLIC_API_BASE_URL.

2. Complete organisation creation - Build and test the form, POST route and organisation list.

3. Complete AI inventory - Register AI systems and display them by organisation.

4. Implement Generate Risks - Create rule-based initial risks and save them into generated_risks.

5. Build the risk dashboard - Show severity counts, scores, categories, regulations and review status.

6. Add review workflow - Allow users to accept, edit, reject or mark risks as treated.

7. Add reports - Generate the AI risk register and compliance summary.

8. Deploy the frontend - Run Next.js permanently and place the platform behind HTTPS and a domain.

9. Add authentication and roles - Super Admin, Org Admin, Compliance Officer, AI Governance Officer, Auditor and Executive Viewer.

10. Introduce Alembic migrations - Manage future schema changes safely and reproducibly.

Appendix A - Quick Reference

Item	Value
Live API root	http://195.216.168.137:8000/
Health	http://195.216.168.137:8000/health
Swagger	http://195.216.168.137:8000/docs
OpenAPI	http://195.216.168.137:8000/openapi.json
Local frontend	http://localhost:3000 or http://localhost:3001
Local backend	http://localhost:8000
VPS backend path	/opt/trustgrc-360-ai/backend
systemd service	trustgrc-api
Database	default_db
Database tables	organizations, ai_systems, generated_risks
Repository	github.com/Soobi1008/trustgrc-360-ai

End of document